

Unit 2: Process Management

Dr. Arun Kumar

Dept. of CSE, NIT Rourkela

Outlines:

- ▶ What is a process?
- ▶ Structure of a process.
- ▶ Process state.
- ▶ Schedulers and Scheduling Queues.
- ▶ Context switching.
- ▶ Process creation and termination.
- ▶ Inter Process Communication.

Process

- ▶ *Process is a program that has initiated its execution.*
- ▶ A program is a *passive* entity; whereas a process is an *active* entity.
- ▶ *System consists of a collection of processes:*
 - ▶ **OS Processes:** *executes systems codes.*
 - ▶ **User Processes:** *executes user codes.*

Note: All these processes may execute concurrently.

Process

- ▶ A process needs certain *resources* like *CPU Time, memory, files* and *I/o devices*.
- ▶ These resources are *allocated* to the process when it is *created* or during *execution*.
- ▶ OS is responsible for:
 - ▶ *Creation & Deletion* of system and user processes
 - ▶ *Scheduling* of the Processes
 - ▶ *Synchronization*
 - ▶ *Communication*
 - ▶ *Deadlock* handling

[SSLD]

Process structure

A process includes:

POS

MVP

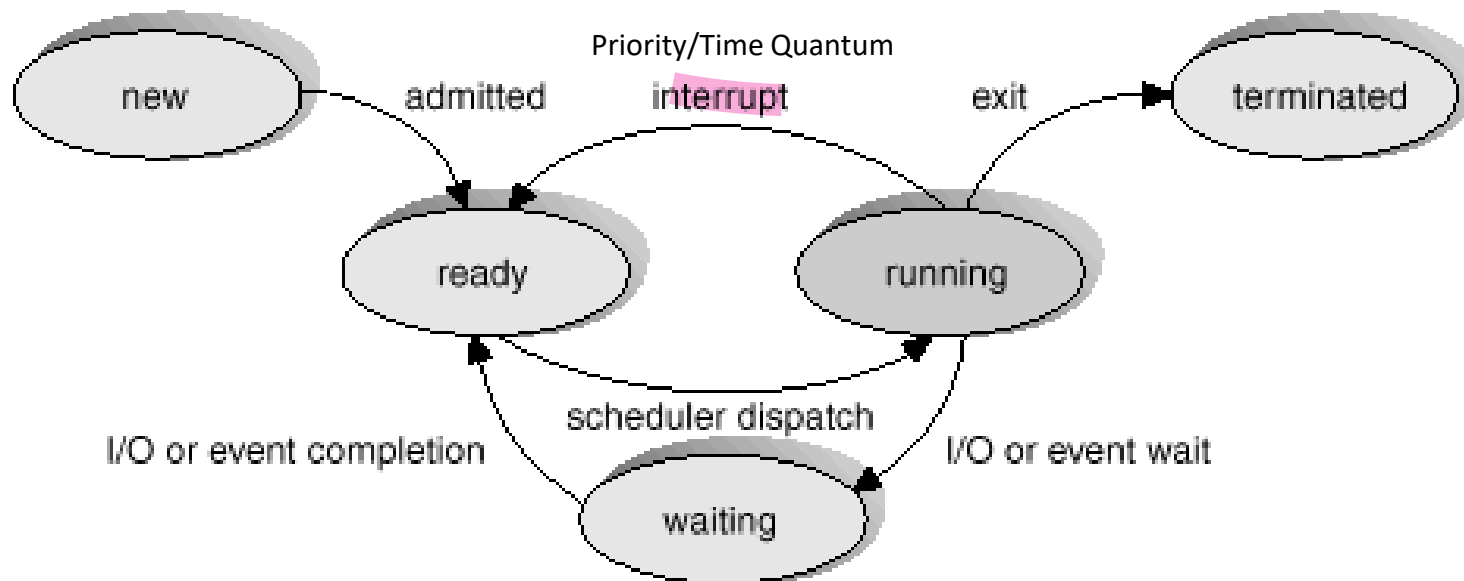
- ▶ *Program section* - contains a copy of the machine code instructions.
- ▶ *User data section* - to hold variable data values.
- ▶ *System data section* - to hold
 - ▶ *Process context information* when process interrupted - program counter, processor registers, etc.
 - ▶ *System information* about resources allocated to process, etc.
- ▶ User and System data sections obviously may be different from one run of a process to another.

Process State

NRRWT

As a process executes, it changes state.

- ▶ **New:** The process is created by OS.
- ▶ **Ready:** The process is waiting to be assigned to a processor.
- ▶ **Running:** Instructions are being executed on processor
- ▶ **Waiting:** The process is waiting for some event to occur
- ▶ **Terminated:** The process has finished execution.

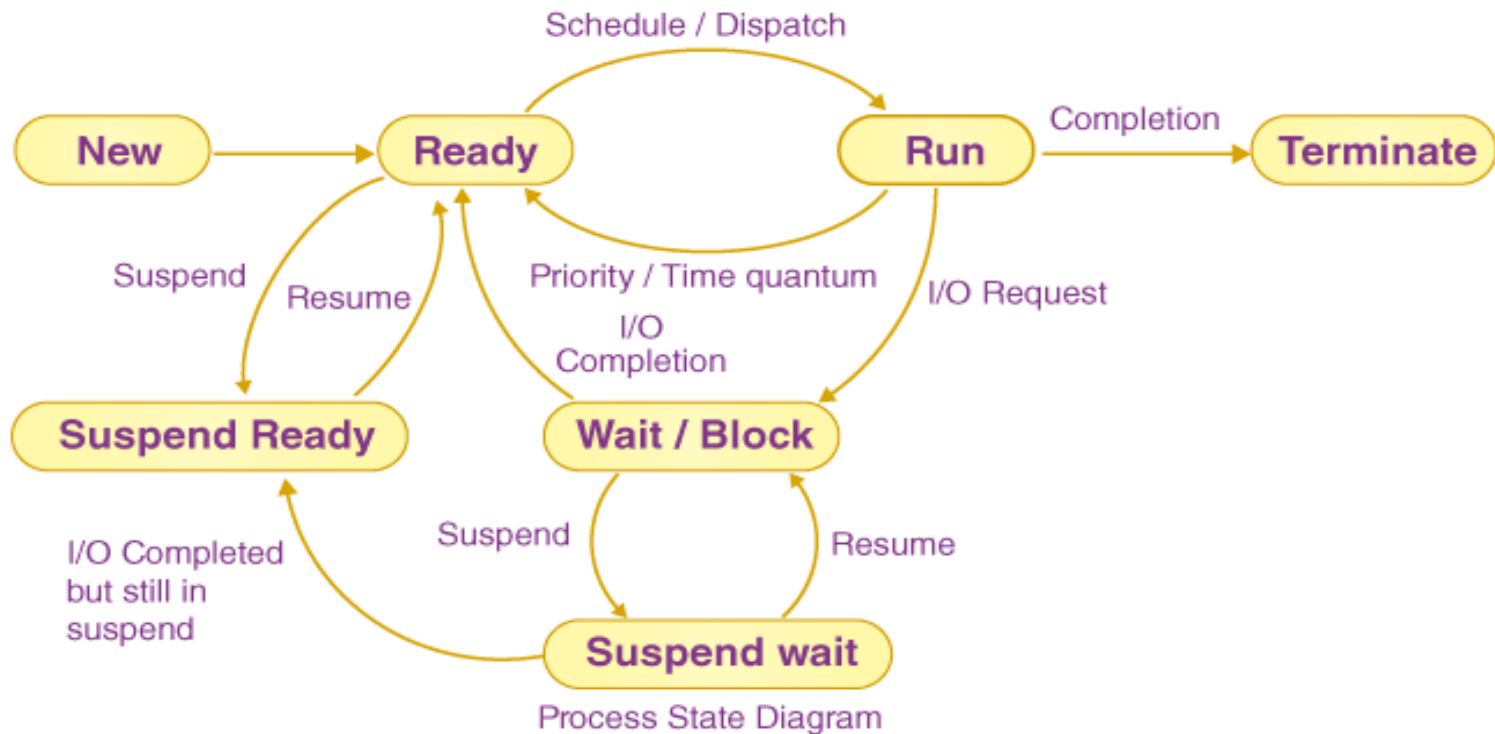


Process States

Process State

Few more states are possible but not standardized.

- ▶ **Suspend wait:** Wait/Block queue is full (Main Memory) then some lower priority processes would be shifted to suspend wait.
- ▶ **Suspend Ready:** Ready queue is full (Main Memory) and a process with a higher priority needs to be executed.

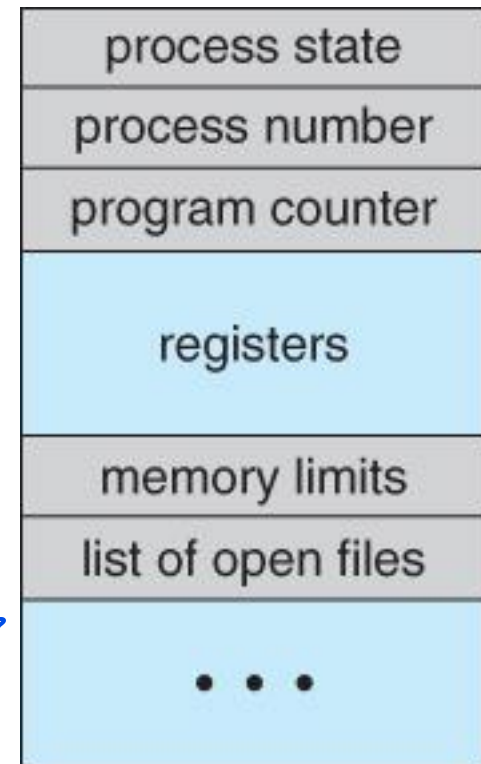


Process States

Process Control Block (PCB)

► PCB - data structure that contains Information associated with each process:

- Process ID
- Process State
- Program Counter
- Register Contents
- Starting address of the process
- Size of the process
- Pointer to Child Process
- Pointer to Next PCB (Sibling Pointer)
- Resource Pointer

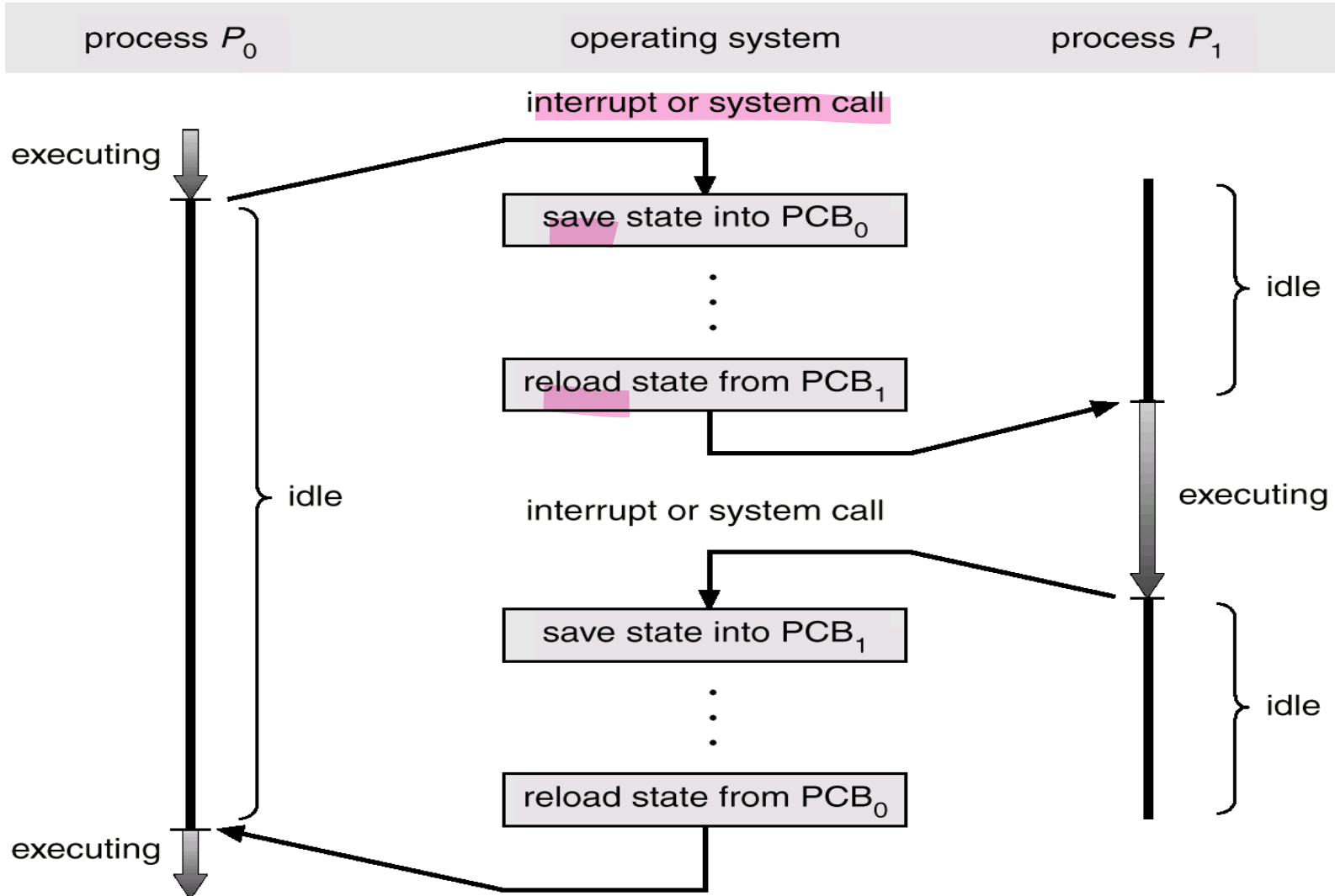


Process Control Block

Context Switch from Process to Process

- ▶ When CPU switches to another process, the system must **save the state** of the old process and load the **saved state** for the new process via a **context switch**
- ▶ **Context** of a process represented in the *PCB*
- ▶ Context-switch time is pure overhead; the system does no useful work while switching
 - ▶ The more complex the OS and the PCB → the longer the context switch
- ▶ Time dependent on hardware support
 - ▶ Some hardware provides multiple sets of registers per CPU → multiple contexts loaded at once.

CPU switched between processes

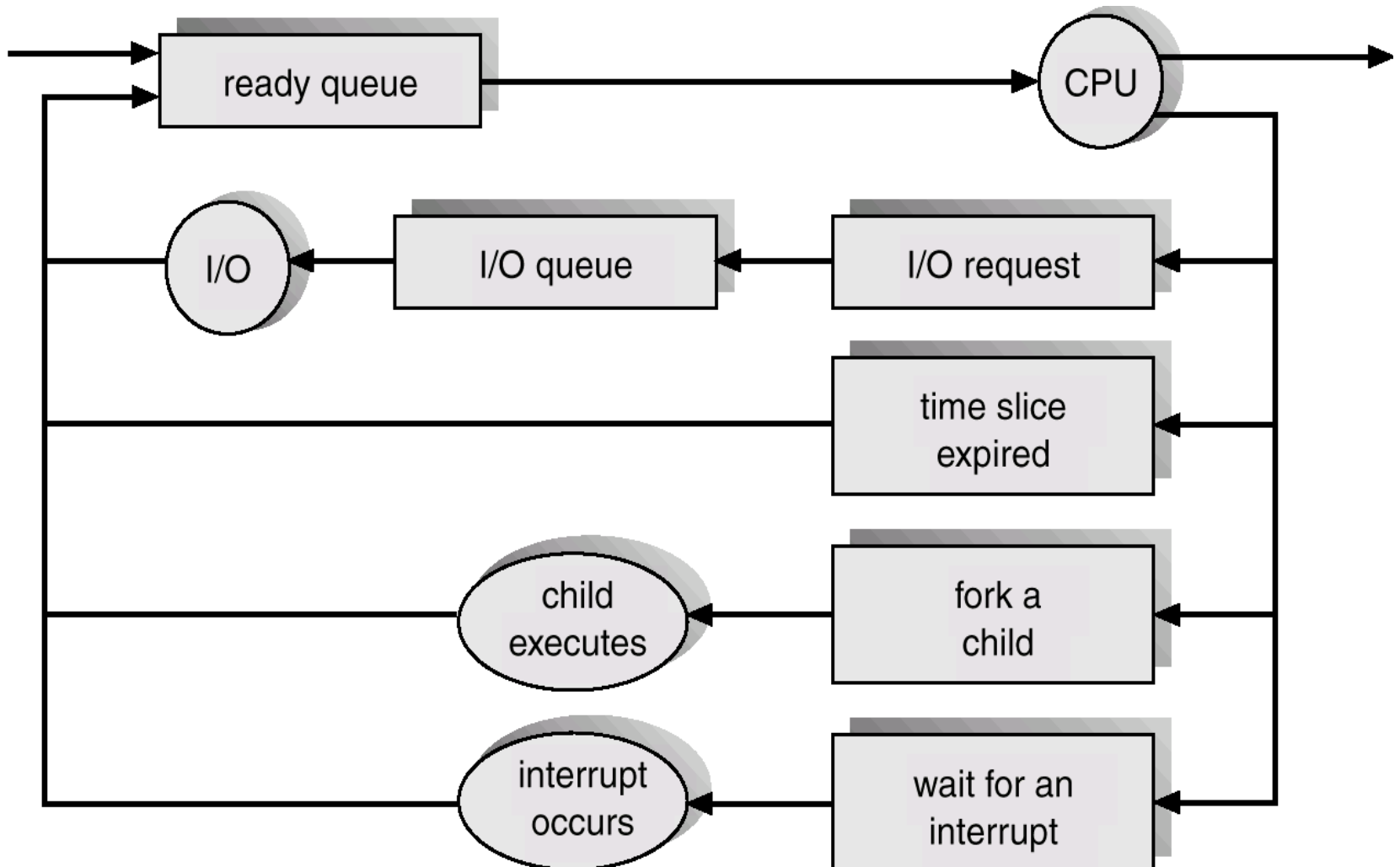


Process Scheduling Queues



- ▶ **Job queue** – Set of all processes just created by LTS (new state processes).
- ▶ **Ready queue** – set of all processes that are in main memory, ready to execute (ready state processes).
- ▶ **Device queues** – set of processes waiting to use an I/O device (waiting state processes).

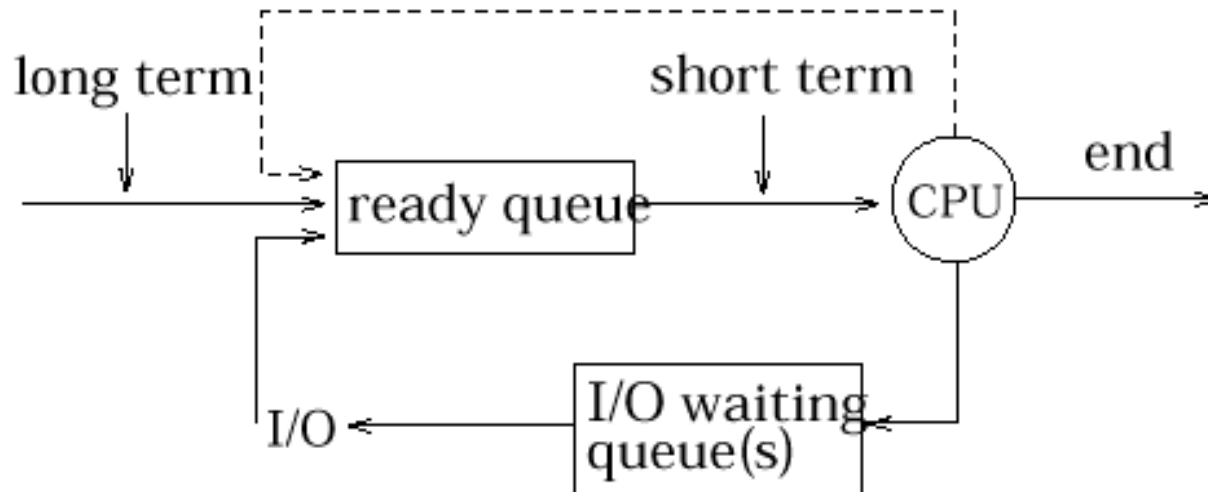
Queueing Diagram



Queueing-diagram representation of process scheduling

Schedulers

- ▶ **Long-term scheduler (LTS)** (or **Job scheduler**) – selects which *processes* should be brought into the *ready queue*.
- ▶ **Short-term scheduler (STS)** (or **CPU scheduler**) – selects which *process* should be executed next on *CPU*.



Schedulers

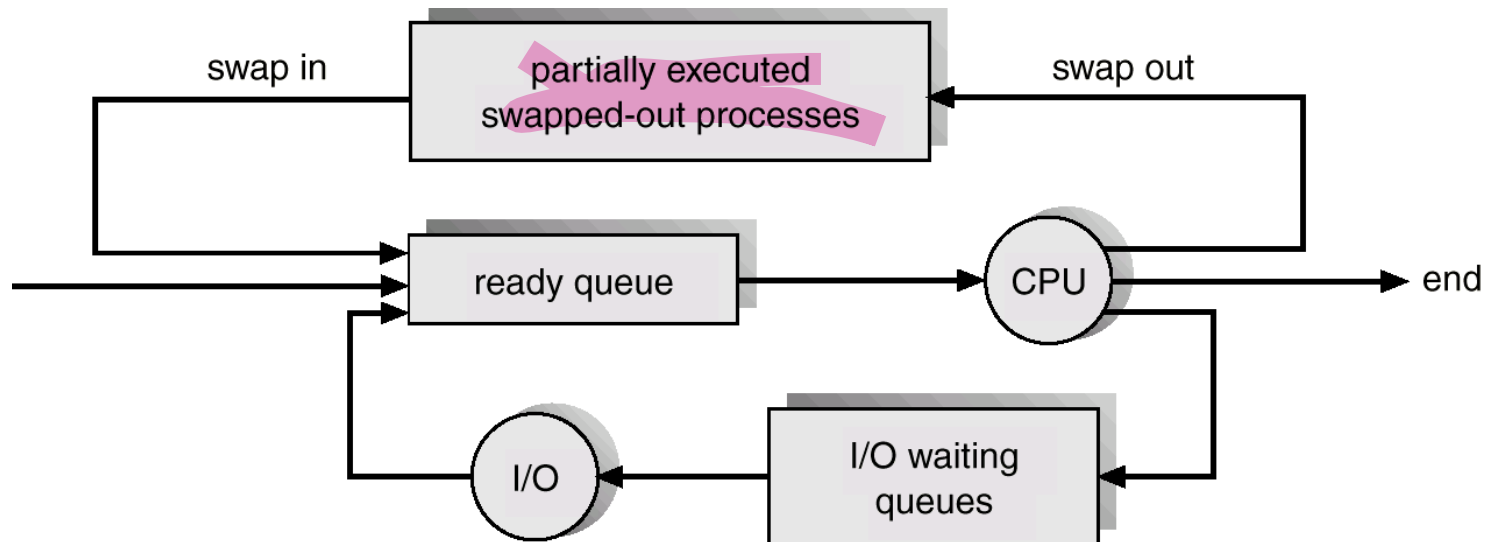
- ▶ ***Short-term scheduler*** is invoked very *frequently* (milliseconds) -> (must be fast).
- ▶ ***Long-term scheduler*** is invoked very *infrequently* (seconds, minutes) -> (may be slow).
- ▶ The ***long-term scheduler*** controls the *degree of multiprogramming* - this is the number of processes in the system that can be scheduled onto the CPU i.e. *number of processes competing for CPU as a resource.*

Schedulers

- ▶ **Long-term scheduler** should choose a **good process mix**; otherwise it may lead to either **low CPU utilization** (if I/O bound processes are selected only) or **low Device utilization** (if CPU bound processes are selected only).
- ▶ **I/O-bound process** – have **small amount of computation** before it needs to do some I/O; many I/O requests - Typical interactive user programs.
- ▶ **CPU-bound process** – **large amount of computation** before it needs to do some I/O; few I/O requests - Programs that require sustained periods of calculation e.g. modelling applications

Schedulers

- ▶ Some OS have a light LTS; and a **Medium Term Scheduler (MTS)**.
- ▶ *If degree of multiprogramming becomes high* then total context switch overhead will be more.
- ▶ MTS can temporarily **swap** some processes out to disk and bring them back later.



Operations on Processes

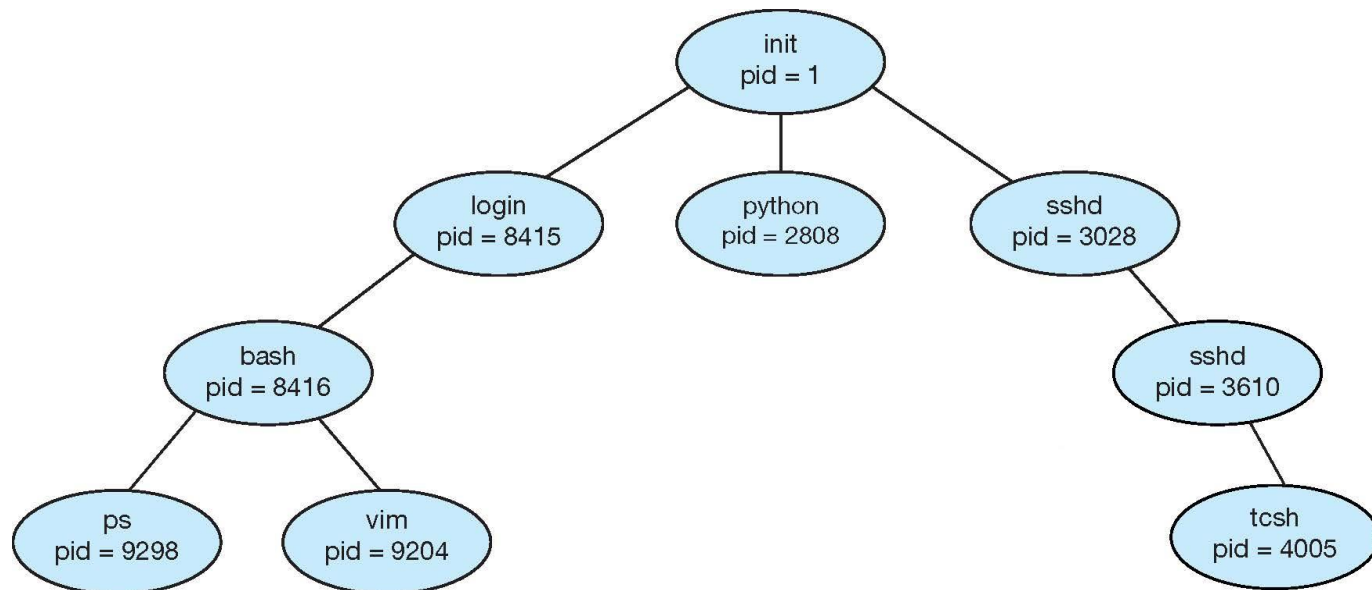
- ▶ The *processes* in the most systems can *execute concurrently*, and they may be *created and deleted* dynamically.
- ▶ Thus, these system must provide mechanisms for:
 - ▶ process *creation*,
 - ▶ process *termination*

Process Creation

- ▶ A **process** may create other processes.
- ▶ **Parent** process create **children** processes, which, in turn create other processes, forming a **tree** of processes.
- ▶ Generally, a process is identified and managed via a **process identifier (pid)**, which is an integer number.
- ▶ The ***init** process* (which always has a ***pid of 1***) serves as the *root parent process for all user processes.*
- ▶ The ***init** process* is created at *bootup of the OS* and is *part of OS.*
- ▶ We use the term **Process** rather loosely, as Linux prefers the term **task** instead.

Process Creation

- ▶ A Tree of Processes in UNIX
- ▶ **Pid** can be used as *an index to access various attributes* of a process (*Process State, memory, CPU etc.*) within the *kernel*.
- ▶ **Example:** sshd process is responsible for *managing clients* that connect the system by using ssh (**Secure shell**)



Process Creation (Cont.)

- ▶ Resource sharing between parent and child processes - options include:
 - ▶ Parent and children *share all resources*, or
 - ▶ Children share *subset of parent's resources*, or
 - ▶ Parent and child *share no resources*.
- ▶ Execution options:
 - ▶ Parent and children *execute concurrently*, or
 - ▶ Parent waits until *children terminate*.

Process Termination

Created when a child process finishes but its parent has not yet read its exit status using `wait()` or `waitpid()`.

Occupies an entry in the process table but uses no CPU or memory, only keeping the exit code.

Becomes a zombie only temporarily; disappears once the parent collects its status or if the parent dies (init/adoptive parent cleans it).

- ▶ Process executes last statement and asks the operating system to *delete it* (***exit*** system call). Then
 - ▶ Return data from *child to parent*.
 - ▶ Process' resources are *deallocated* by operating system.
- ▶ Parent may terminate execution of children processes (***abort***). Reasons for this could be:
 - ▶ The child has *exceeded its usage of some of the resources* that has been allocated. (Parent should have a mechanism)
 - ▶ The task assigned to child is *no longer required*.
 - ▶ Parent is exiting and OS does not allow children to continue if their parent terminates – ***cascaded termination***.

▶ Zombie Process?

Background process that runs independently of any terminal (e.g., cron, sshd).

Started at boot or via services, performing continuous system tasks without user interaction.

Detached from controlling terminal, usually with parent as `init` or `systemd`; runs with its own session and no user input.

Inter-process Communication

- ▶ Processes within a system may be *independent* or *cooperating*
- ▶ Process is *independent* if it cannot legally affect or be affected by the execution of another process.
 - ▶ Any process which does not share data with another process is *independent process*.
- ▶ Process is *cooperating* if it can legally affect or be affected by the execution of another process.
 - ▶ Any process which shares data with another process is *cooperating process*.

1 L L m

Why Cooperating Processes?

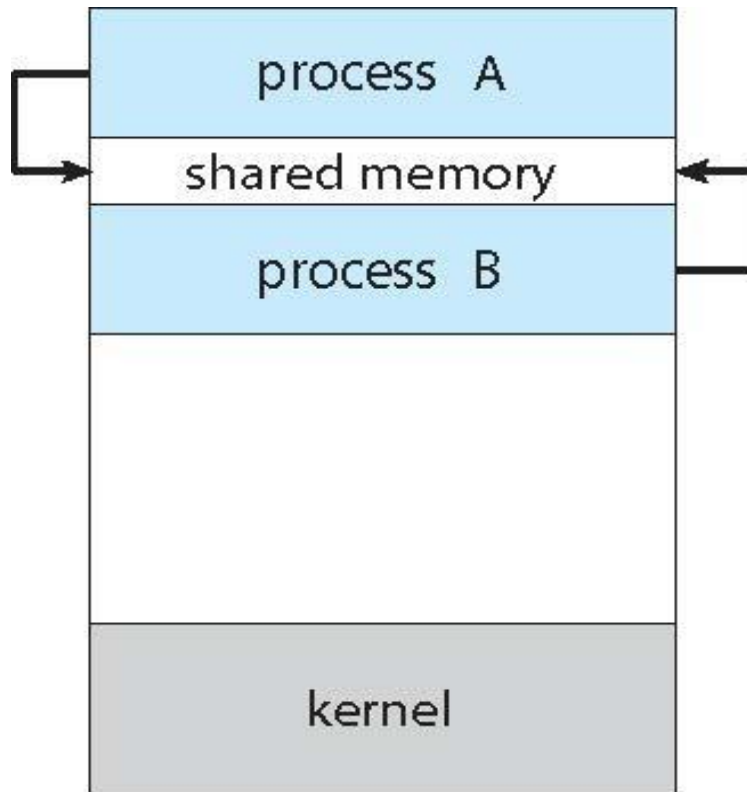
- ▶ **Information sharing:** Processes may need to share data
 - ▶ More than one process *reading/writing the same data* (a shared file, a database record,...)
 - ▶ *Output of one process being used by another*
- ▶ **Computation speed-up:** If we want *a task to run faster*, we must *break it into subtasks*, each of which will be *executing in parallel* but only if *>1 processor*.
- ▶ **Convenience:** A user may work on many task at the same time.
 - ▶ A user may be editing, listening to music and compiling in parallel.
- ▶ **Modularity:** we want to construct the system in a modular fashion, *dividing system functions into separate processes that then talk to each other.*

Inter-Process Communication (IPC)

- ▶ Cooperating processes need **interprocess communication (IPC)** to exchange data and information.
- ▶ There are two fundamental types of IPC
 - ▶ ***Message passing:***
 - ▶ P and Q exchange messages
 - ▶ ***Shared memory:***
 - ▶ P writes into a shared location, Q reads from it and vice-versa

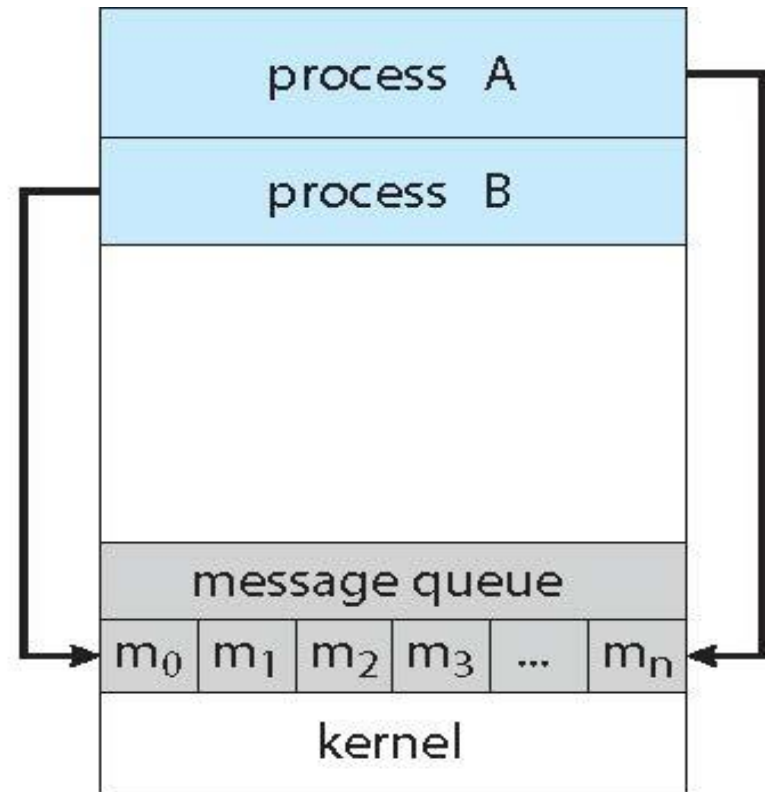
Inter-Process Communication (IPC)

(a) Shared Memory



(a)

(b) Message Passing



(b)

Inter-Process Communication (IPC)

- ▶ Both models are common in OS.
- ▶ *Message passing* is useful for exchanging the *smaller amount of data*.
- ▶ *Message passing* is also easier to implement in *distributed system* than shared memory.
- ▶ *Shared memory* can be *faster* than message passing, since message-passing systems are typically implemented using *system call* and thus require the *more time-consuming task* of *kernel intervention*.

Message Passing System

- ▶ To pass message, a **communication link** must exist between the Processes (Ex: Processes P & Q).
- ▶ This *Communication Link* can be **implemented using a variety of ways**, there are several methods for *logically implementing a link* and the **send()/receive()** operations like:
 - ▶ Direct vs. Indirect
 - ▶ Symmetric vs. Asymmetric
 - ▶ Synchronous vs. Asynchronous

Message Passing System: Direct

Naming: *Processes* that wants to communicate must have *a way to refer to each other*. Ex: Process **P** and Process **Q**.



Primitives:

- ▶ **Connect** (sender address, receiver address), for connection-oriented communication.
- ▶ **Send** ([receiver], message) e.g., Send(P, msg) – *Send a message to Process P.*
- ▶ **Receive** ([sender], message) e.g., Receive(Q, msg) – *Receive a message from Process Q.*
- ▶ **Disconnect** (connection identifier), for connection-oriented communication.

Message Passing System: Direct

- ▶ **Symmetric (in addressing)**– Both the *sender and receiver* have to *explicitly name each other*.

- ▶ Send(P, msg)
- ▶ Receive(Q, msg)

- ▶ **Asymmetric (in addressing)** – Only the *sender has to name* the *receiver*, but the *receiver doesn't need to name the sender*.

- ▶ Facilitates *one-to-many communication*.
- ▶ Send(P, msg); - *send message to P.*
- ▶ Receive(id, msg); - *receive message from any process. “id” will be set to the name of the process with which communication has taken place.*

Problem with Direct Communication:

- ▶ The **Disadvantage** in both these schemes (**Symmetric** and **Asymmetric**) is the *limited modularity* of the resulting process identifiers.
- ▶ Changing the *identifiers of a process* may necessitates examining *all other process definitions*.
- ▶ It means that *if a name of a process is changed then it will require the changes in many processes.*

Message Passing System: Indirect

- ▶ The messages are sent and received to/from **mailboxes or ports**.
- ▶ A **mailbox** can be viewed abstractly as *an object* into which *messages can be placed* by processes and from which *messages can be removed*.
- ▶ Each mailbox has a *unique identifier*.
- ▶ Two processes can communicate with each other if they have a *shared mailbox*.
 - ▶ Send (A, msg) – Send a message to mailbox A
 - ▶ Receive (A, msg) – Receive a message from mailbox A

Message Passing System: Indirect

Link Properties:

- ▶ *A link is established* between a pair of processes only if both member of the pair have *a shared mailbox*.
- ▶ A link may be associated with *more than two processes*. (remember in *direct* communication link must be *only between two processes*)
- ▶ Between *each pair of communicating processes*, there may be a number of *different links*, with each link corresponding to *one mailbox*.

Message Passing System: Indirect

Ex: Process P1 send a message to A (mailbox), while two processes P2 and P3 execute receive() from A. Which process will receive the message send by P1?

Message Passing System: Indirect

The answer depends on which of the following methods we choose:

- ▶ Allow a *link* to be associated with *two processes at most*.
- ▶ Allow *at most one process* at a time to execute a *receive() operation*.
- ▶ Allow the system to *select arbitrarily* which process will receive the message (that is *either P2 or P3* , but not both, will receive the message). The system also may define *an algorithm for selecting* which process will receive the message (that is *round robin* where processes take turns to receive messages). The system may identify the the receiver to the sender.

Message Passing System: Synchronous Vs. Asynchronous

- ▶ Synchronous – After initiating a send()/receive() operation, the *process waits for the acknowledgement* (the process is *blocked*)
- ▶ Asynchronous – After initiating a send()/receive() operation, the process *continues its execution*.

Message Passing System: Synchronous Vs. Asynchronous

- ▶ **Blocking send:** The *sending process is blocked* until the *message is received* by the *receiving process or by the mailbox*. **(zero buffer)**
- ▶ **Nonblocking send:** The sending process *sends the message and resumes operations*. **(Bounded or unbounded buffer)**
- ▶ **Blocking receive:** The receiver blocks until a message is *available*. **(zero buffer)**
- ▶ **Nonblocking receive:** The receiver *retrieves either a valid message or a null*. **(Bounded or unbounded buffer)**

Rendezvous – sender and receiver both are blocking

Buffering during message passing

► Zero capacity

- No waiting queue/buffer.

- *Sender must wait for receiver, after generating one item.*

► Bounded capacity

- Buffer has a fixed length N .

► Unbounded capacity

- No practical limit on the size of the buffer

Process: Exception Conditions

► Process Termination

- **Sender Terminates:** If the receiver has initiated a blocking receive then the OS either terminates the receiver or notifies it that Sender has terminated.
- **Receiver Terminates:** If the sender has initiated a blocking send then the OS either terminates the Sender or notifies it that Receiver has terminated.

► Lost Messages

- The OS is responsible for detecting this event and for resending the message.
- The Sender is responsible for detecting this event and for resending it.
- The OS detects the event and notifies the Sender about it.

► Scrambled Messages

- Due to noise in the channel
- Handled in the similar ways as lost messages.